

Care Plan Integration Guide

Which screen calls which endpoint, what comes back,
and exactly what to show the doctor and the patient.

DATE	7 October 2026
API BASE	https://api.momcare.solutions/api/
SWAGGER	https://api.momcare.solutions/api/docs/
POSTMAN	folders: Care Plans, Care Plan Preferences

The plan text is AI-written and not yet doctor-approved

Always show the label, the sources and the generated-by-AI notice exactly as described here.
A doctor reviews every medium / high-risk plan before it counts as reviewed.

Start here

Four screens use the care plan. **Almost everything is one call:** GET

/api/pregnancies/{pregnancy_id}/current-care-plan/ returns the whole plan (nutrition, exercise, progress, medications, notes, allergies, sources). You do **not** call nutrition and exercise separately unless a screen really needs only one of them.

Screen	Who	Calls
A. Doctor dashboard — patients needing a plan review / patients missing a plan	Staff	GET /patients/dashboard-kpis/ · GET /patients/?workflow=care_plan_review · ?workflow=care_plan_missing
B. Patient page → Care plan tab (staff view, editable)	Staff	GET current-care-plan/ · then the edit endpoints in Part 4
C. Patient mobile app → Care plan page (read only)	Patient	GET current-care-plan/ only (the same URL; staff-only fields are removed for her)
D. Hospital settings → Care plan preferences	Hospital admin	GET/PATCH/POST /care-plan-preferences/...

Auth, roles and errors

- Every call: Authorization: Bearer <access token> from POST /api/auth/login/.
- **Staff** (provider, nurse, care manager, hospital admin) can read any plan of their own hospital and write notes, edit items, set allergies and mark *reviewed*. **Only a provider** writes medications. **Finalize / reopen:** provider or hospital admin. **Preferences:** hospital admin only. A **patient** can only read her own plan.
- A plan of another hospital is **404** (never 403). A missing permission is **403**. Editing a **finalized** plan is **409** (reopen it first). Bad input is **400** with a message.
- Ids are UUID strings. Dates are ISO (2026-10-06); times are UTC ISO with a Z.

How do I get pregnancy_id? **Staff:** each row of GET /api/patients/ carries pregnancy_id, and the patient detail carries current_pregnancy.id. **Patient app:** GET /api/auth/me/ returns patient_id and pregnancy_id (null if she has no active pregnancy) — **only for a patient account**; staff and admin responses are unchanged. Login is unchanged too.

The one call: current care plan

GET /api/pregnancies/{pregnancy_id}/current-care-plan/ — staff and the patient. It returns the plan for the **week she is in today** (a week belongs to the 30-day monthly plan that holds its first day).

```
{
  "care_plan": { ...the plan, see Part 3... } // null until her first reading creates one
  "preparing": false, // she has readings but this week's plan is not written yet
  "update_pending": false, // the plan shown has not yet looked at her newest reading
  "status_message": null // ready-to-show text when either flag is true
}
```

What to do with the three states

Response	Meaning	Show
care_plan null, preparing false	She has no readings yet . Nothing is on its way.	Patient: "Your plan will appear after your first reading." Staff: the patient is in the <i>Care plan missing</i> workflow; prompt to take a first reading.
preparing true	Readings exist, the week's plan is being written (a few seconds, or up to 15 min if the job is retrying).	status_message; poll the same URL every 3-5 s until it clears.
update_pending true	A plan is shown but a newer reading is still being evaluated (about 10-20 s).	The plan, with a small banner status_message; poll every 3-5 s.

Today in production the plan is written inside the reading's request, so a reading that starts a plan can take 10-20 seconds to save and preparing is rarely seen. A background worker may be switched on later; build the polling now and nothing changes for you then.

Other ways to read a plan

Endpoint	Use
GET /pregnancies/{id}/current-care-plan/nutrition/ and /exercise/	Only if a tab shows one section alone. Returns that section (content, progress, sources, label...) plus week, status, disclaimer and warning signs.
GET /care-plans/?pregnancy={id}	History: every monthly plan of a pregnancy, newest first (paginated).
GET /care-plans/{plan_id}/	One monthly plan by id (the same plan object, without the three flags above).

What is in the plan and how to show it

Field	Show it as
month_number, weeks_label, trimester, week.week_number / week_start / week_end	Header: "Month 5 · weeks 17-21 · Trimester 2 · This week: week 20 (6-12 Oct)".
message	One line under the header ("Keep following it; we will tell you if anything needs to change").
status_in_progress reviewed finalized	Staff badge. The patient never needs it.
nutrition.label / exercise.label	Always show. suggested_automatically → "Suggested automatically"; reviewed_by_provider → "Reviewed by your provider".
progress.text	A paragraph at the top: how she did last week (all text is written by the backend).
nutrition.progress.text / exercise.progress.text	The same, but about that section's own vitals (nutrition: hemoglobin, glucose, blood pressure; exercise: heart rate, activity, stress, blood pressure). Put each inside its section. .facts holds the numbers if you chart them.
contact_care_team + contact_message	When true (her latest reading is high risk): a red banner at the very top: "Please contact your care team today."
reading_advice (object or null)	A short card "Right now": tips list, plus its sources. It never replaces the weekly plan. Hide it when null.
weeks[]	Optional history strip: each week with its trend (improved / steady / worse / null).

Nutrition and exercise content

Section	content keys	Notes
Nutrition	meals[] {slot, text, item_key, source}; foods_to_eat[], foods_to_avoid[] {text, item_key, source}; hydration (text); timing_tips[] (texts); weekly_rotation (always empty - ignore)	slot = breakfast snack lunch dinner. Up to 6 meals, 6 + 6 foods, 3 tips. About 130-300 words in all.
Exercise	activities[] {text, duration_minutes, frequency_per_week, intensity, item_key, source}; avoid[] {text, item_key, source}; stop_signs[] (texts)	intensity = light moderate. Up to 4 activities, 4 avoid, 5 stop signs. About 30-160 words.

Every item has source: **ai** (written by the AI) or **staff** (a doctor added or edited it). Show a small "edited by staff" mark on **staff** items. item_key is an id for editing, **never display it**.

Sources: the rule that matters most

basis	Meaning	What you must show
web_search	The AI searched the web for the official guidance of the patient's country; source_links[] {title, url, host} are the pages it used.	A "Sources" list of links (open in a new tab). Never hide it.
ai_only	No official document was found. The plan is the AI's own work.	generated_by_ai_notice prominently , in the section. No links.
baseline	The AI failed; MomCare's fixed generic plan.	The sources[0] line ("MomCare generic safe baseline plan..."); staff should review.

`sources_verified` is always false: the pages are real, but nobody has checked that the advice follows them. Do not present the sources as a guarantee.

The rest of the plan

Field	Show it as
medications[] {id, text, added_by, created_at}	"Medication (from your provider)". Written only by doctors — the AI never writes medicines. Patient sees it read-only.
notes[] {id, text, added_by, created_at}	"Notes from your care team". Always visible to the patient.
allergies_and_conditions {food_allergies[], dietary_preference, conditions[{field,label}]}	"This plan was built around...". Staff can edit it (Part 4). dietary_preference: none vegetarian vegan other.
disclaimer, warning_signs[]	Always show both at the bottom of the patient's page, unchanged.
reviewed_by/at, finalized_by/at, last_evaluated_at, clinical_review	Staff only (the patient does not receive them). "unreviewed" = the rules behind the plan are not clinically signed off.

A plan with doctor edits

Staff edits are layered on top of what the AI wrote when you read the plan. A removed item is simply missing; an edited or added one has `source: staff` and an `adjustment_id`. When the AI writes a new weekly plan, the doctor's edits are **kept**. After any staff edit a *reviewed* plan goes back to *in_progress* (what was reviewed has changed).

PART 4

Staff actions (screen B)

All of these return the **whole updated plan** (same shape as Part 3), so after any call just replace your local plan with the response.

Action	Request	Who
Add / edit / remove an AI item	POST /care-plans/{id}/adjustments/ body {section, list_name, action, item_key, content}	staff
Change an adjustment you made	PATCH /care-plans/{id}/adjustments/{adj}/ body {content}	staff
Undo an adjustment (a removed AI item comes back)	DELETE /care-plans/{id}/adjustments/{adj}/ body {reason?}	staff
Add a medication	POST /care-plans/{id}/medications/ body {text}	provider
Edit / remove a medication	PATCH body {text} · DELETE body {reason?} on /care-plans/{id}/medications/{med}/	provider
Add a note	POST /care-plans/{id}/notes/ body {text}	staff
Edit / remove a note	PATCH body {text} · DELETE on /care-plans/{id}/notes/{note}/	staff
Allergies, diet, conditions	PATCH /care-plans/{id}/allergies-conditions/ — send only what changed	staff
Mark reviewed	POST /care-plans/{id}/review/	staff
Finalize / reopen	POST /care-plans/{id}/finalize/ · /reopen/	provider or admin

Rules the server enforces (show its message on 400 / 409): an edit may not mention medicines, supplements, doses or calories (400). Only a provider writes medications (403). A finalized plan cannot be edited (409 — offer "Reopen"). Allergies edited here update the patient's record everywhere.

Bodies you will actually send

```
// remove an AI item
POST /care-plans/{id}/adjustments/
{ "section": "nutrition", "list_name": "meals", "action": "remove", "item_key": "oats" }

// replace one
{ "section": "exercise", "list_name": "activities", "action": "edit", "item_key": "walking",
  "content": { "text": "Slow walk after dinner", "duration_minutes": 15,
               "frequency_per_week": 5, "intensity": "light" } }

// add a new one
{ "section": "nutrition", "list_name": "foods_to_eat", "action": "add",
  "content": { "text": "Chana: protein and iron" } }

// allergies (this also re-plans the week straight away; the response can take 10-20 s)
PATCH /care-plans/{id}/allergies-conditions/
{ "food_allergies": ["peanut", "egg"], "dietary_preference": "vegetarian", "diabetes": "yes" }
```

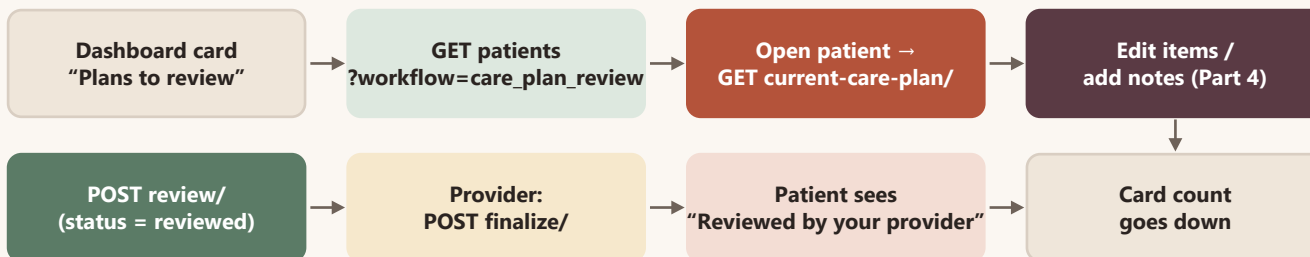
list_name allowed values	
section nutrition	meals · foods_to_eat · foods_to_avoid
section exercise	activities · avoid
condition fields (values yes no unknown)	previous_c_section · previous_preeclampsia · previous_gestational_diabetes · previous_preterm_birth · chronic_hypertension · diabetes · multiple_pregnancy

Doctor dashboard workflows (screen A)

Two lists tell a doctor what needs attention. They are query parameters on the normal patient list, not separate endpoints.

Call	Returns
GET /api/patients/?workflow=care_plan_review	Patients whose current plan is in progress and medium or high risk . Low-risk plans do not need a doctor. Open each patient's plan and press <i>Review</i> .
GET /api/patients/?workflow=care_plan_missing	Active pregnancies with no plan for this week : a brand-new patient with no readings (prompt staff to take a first reading) or a plan that failed. A plan appears by itself after her first reading; the sweep job retries failures every 15 min.
GET /api/patients/dashboard-kpis/	Counts for the cards: workflow.care_plan_review and workflow.care_plan_missing (next to risk_review, low_confidence and the care_activities).

A doctor's review flow



Each patient-list row carries pregnancy_id; use it for the plan URL. The row also has the usual filters (?location=, ?assigned_to=me, ...).

Hospital preferences (screen D, hospital admin only)

When **three different staff members** make the same correction (same food, same region and trimester), the system suggests a hospital-wide preference. The admin approves or rejects it; an approved one is passed to the AI for that hospital's future plans. It is per hospital and never shared. An optional screen.

Endpoint	Use
GET /care-plan-preferences/?status=suggested approved rejected inactive	The admin's inbox (paginated).
GET /care-plan-preferences/{id}/ · PATCH body {guidance}	Read one; reword a suggestion before approving.
POST /care-plan-preferences/{id}/approve/ · /reject/ · /deactivate/	Decide. Optional body {guidance} on approve. Deactivate switches an approved one off.

Build checklist and things to know

Per screen

- **Patient app (C):** read-only. One GET; poll while preparing / update_pending. Show label, sources or the AI notice, medication, notes, disclaimer and warning signs. Hide staff-only fields (you will not receive them).
- **Staff plan tab (B):** the same plus Part 4 buttons per item (remove / edit / add), notes, medication (providers only), allergies editor, *Review*, *Finalize*, *Reopen*. Disable edits when status is finalized.
- **Dashboard (A):** two cards from dashboard-kpis, two lists from the patient list parameters.
- **Always re-render from the response** of a write call; do not patch local state by hand.

Things that will surprise you

- A new patient has **no plan until her first reading**; care_plan is null and that is normal.
- Saving allergies (or a reading that starts a plan) can take **10-20 seconds**: show a spinner, set a request timeout above 30 s.
- contact_care_team can be true while reading_advice is null (the weekly plan was already written at high risk). Show the banner from contact_care_team, not from the advice card.
- A week starts at **12 am in the hospital's time zone**. Use week_start / week_end as given; do not recompute weeks on the client.
- Text fields are plain text. Never render them as HTML.
- clinical_review: "unreviewed": a doctor must review plans in the dashboard workflow; do not describe a plan as clinically approved unless label is reviewed_by_provider.

Backend items still open (ask before relying on them)

- Background plan writing (the "being prepared" state) is built but switched off in production until a worker is deployed.
- The doctor review of the AI rules (risk limits, banned words, exercise caps) is pending.

Swagger (/api/docs/, admin login) and the Postman folders **Care Plans** and **Care Plan Preferences** contain every request and sample response above.